APOL: When Does Learned Sentence-Level Compression Help in Retrieval-Augmented Question Answering?

Kissa Zahra
NUCES
Islamabad, Pakistan
i210572@nu.edu.pk

Zaib Un Nisa
NUCES
Islamabad, Pakistan
i210383@nu.edu.pk

Kalsoom Tariq
NUCES
Islamabad, Pakistan
i212487@nu.edu.pk

Muhammad Atif Saeed
NUCES
Islamabad, Pakistan
atif.saeed@isb.nu.edu.pk

Abstract

Retrieval-augmented generation (RAG) systems commonly pass all retrieved sentences to a language model without filtering, which adds noise and inflates inference cost. We present APOL, an exploratory evaluation protocol that compares heuristic and learned sentence-level compression for multi-hop question answering under a fixed reader and retrieval setup. Its learned component, the *Adaptive Hybrid Compression Controller* (AHCC), is a DistilBERT sentence classifier trained on automatically generated labels, with no human annotation, that assigns KEEP, DROP, or EMBED to retrieved sentences and feeds an extractive assembly policy. On a 100-question HotpotQA sample, every compression method in the main comparisons has a higher observed Token F1 than the uncompressed prompt of its own comparison, with shorter prompts. In Comparison A, AHCC reaches Token F1 0.181 versus 0.168 for the full-pool baseline at 47.3% mean per-question compression, and BM25-threshold pruning reaches 0.217 at 51.2%. Two AHCC ablation variants, and AHCC under tail-noise injection, fall below the uncompressed prompt, and no difference has been tested for statistical significance. With the classifier fixed, replacing abstractive T5 assembly of the EMBED class with extractive top-1 selection raises pipeline F1 from 0.137 to 0.171, so the assembly step, and not only classifier accuracy, affects pipeline quality in this setup. In a stored pool of ten sentences per question, going from five to ten sentences lowers the uncompressed F1 from 0.209 to 0.168, while the compressed pipelines change by 0.004 or less; larger depths were not tested. A confidence-threshold sweep moves AHCC between 41% and 63% compression, with Token F1 varying non-monotonically between 0.167 and 0.181. The AHCC training pool contains the evaluation questions, so all AHCC results are in-pool and do not establish generalisation to unseen questions.

1 Introduction

Retrieval-augmented generation (Lewis et al., 2020; Gao et al., 2023) has emerged as a widely adopted strategy for grounding language model outputs in external evidence, particularly for knowledge-intensive tasks such as open-domain question answering. A standard pipeline retrieves the top-k passages via a sparse or dense retriever and concatenates them verbatim into the prompt before generation. While straightforward to implement, this approach has two drawbacks: irrelevant retrieved context can degrade model performance (Shi et al., 2023), and models use information in long contexts unevenly (Liu et al., 2024). Long prompts also increase inference latency and cost.

Prompt compression has been proposed as a remedy, with methods operating at the token or lexical-unit level (Jiang et al., 2023b, 2024; Pan et al., 2024; Li et al., 2023) or at the sentence level (Xu et al., 2024; Wang et al., 2023; Hwang et al., 2025; Chirkova et al., 2025). However, many evaluations report a limited set of retrieval conditions, which makes it harder to see when compression helps, how different approaches compare under varying retrieval pool sizes and noise levels, and which failure modes are specific to learned versus heuristic compression.

This paper investigates these questions through a multi-phase exploratory study. The evaluation is limited by question overlap between the classifier-development pool and the pipeline evaluation set, and should not be interpreted as a clean held-out test. We ask: *Does learned adaptive sentence-level compression offer advantages over static BM25-*

score-based pruning in terms of robustness to retrieval-pool changes and token efficiency, and under what conditions does each approach break down?

Contributions.

1. **APOL**: a five-phase evaluation protocol covering baseline establishment, heuristic compression, learned compression, robustness testing under distractor injection and noise, and efficiency analysis, all under the same reader (LLM) and retrieval pool so that methods are compared on identical inputs within each prompt template.
2. **AHCC**: a DistilBERT-based sentence classifier trained on 3,000 automatically generated KEEP/DROP/EMBED labels, extended via self-training on 956 high-confidence pseudo-labels. It reaches a test macro-F1 of 0.603 against its heuristic labels, drives a pipeline with Token F1 0.181 at 47.3% compression (in-pool), and exposes a confidence threshold that sets the compression level after training (41–63%).
3. **Finding 1 — Assembly matters for a fixed classifier**: with the Stage 1 classifier fixed, replacing T5 abstractive EMBED assembly with extractive top-1 selection moves pipeline F1 from 0.137 to 0.171; the binary KEEP/DROP variant scores 0.165, a difference from 0.171 that is too small to interpret. Classification agreement with heuristic labels therefore does not by itself determine downstream QA quality. The 0.034 gap is of the same order as the marginal 95% half-widths (about 0.05) and has not been tested for significance.
4. **Finding 2 — Limited sensitivity to pool size between $k = 5$ and $k = 10$**: the F1 of every compression method changes by 0.004 or less, while the uncompressed baseline falls from 0.209 to 0.168. The pool contains only ten sentences per question, so larger retrieval depths were not tested. In the robustness probes, compressed prompts were shorter than uncompressed prompts, but accuracy did not uniformly exceed the uncompressed baseline (Section 6.4), so the results do not establish general robustness to noise.

2 Related Work

Retrieval-augmented generation. Lewis et al. (2020) established the foundational RAG framework combining dense retrieval with generative language models. Izacard and Grave (2021) showed that fusing evidence from multiple retrieved passages in the decoder improves open-domain question answering as more passages are used. Recent work has studied when retrieved evidence helps versus hurts: Mallen et al. (2023) showed that retrieval can be unnecessary or harmful for popular facts, Shi et al. (2023) demonstrated that irrelevant context substantially degrades performance, and Liu et al. (2024) showed that models use information in long contexts unevenly. Asai et al. (2024) proposed Self-RAG, which trains models to decide dynamically whether to retrieve, generating critique tokens alongside answers.

Prompt and context compression. LLMLingua (Jiang et al., 2023b) compresses prompts at the token level using a small LM as a perplexity oracle; LongLLMLingua (Jiang et al., 2024) makes this question-aware for long contexts, and LLMLingua-2 (Pan et al., 2024) adds task-agnostic data distillation from GPT-4. Selective Context (Li et al., 2023) removes lexical units of low self-information. RECOMP (Xu et al., 2024) trains extractive and abstractive compressors using supervision derived from the downstream LM, and FILCO (Wang et al., 2023) trains a context filter for generation. More recent sentence- and passage-level methods include EXIT (Hwang et al., 2025), which classifies sentences with awareness of their document context; Provence (Chirkova et al., 2025), which casts sentence-level pruning as sequence labelling and unifies it with reranking; CompAct (Yoon et al., 2024), which compresses retrieved documents iteratively for multi-hop QA; and AttnComp (Luo et al., 2025), which uses LLM attention to select context. AutoCompressors (Chevalier et al., 2023) and PISCO (Louis et al., 2025) instead compress context into learned soft representations.

Positioning this work. Table 1 summarises the closest methods. We are not aware of a study that compares heuristic score thresholding with a learned three-way sentence classifier while jointly varying retrieval pool size, distractor count, noise fraction, and confidence threshold under one fixed reader. APOL provides this controlled comparison with a deliberately lightweight learned scorer, and it surfaces a possible failure mode (a gap between classifier agreement and pipeline quality under ab-

stractive assembly) that may not be apparent in single-condition evaluations. Our contribution is empirical; we do not re-run the published compressors above, which are the natural next comparison.

3 Problem Formulation

Let q be a question and $S = \{s_1, \ldots, s_k\}$ be the set of k sentences retrieved by BM25 (Robertson and Zaragoza, 2009) from a per-question document set. A compression policy $\pi(q, s_i) \rightarrow \{\textsc{Keep}, \textsc{Drop}, \textsc{Embed}\}$ assigns one of three actions to each sentence: KEEP sentences are retained verbatim as primary evidence; DROP sentences are excluded; EMBED sentences are included as secondary context. The assembled prompt $P(q, S, \pi)$ is passed to a language model $\mathcal{M}$ to produce answer $\hat{a} = \mathcal{M}(P)$. The goal is to maximise Token F1$(\hat{a}, a^*)$ while minimising $|P|$ in tokens, where a^* is the gold answer.

4 Method

4.1 Heuristic Compression Baselines

We implement BM25-score-based baselines in two settings. In the main comparison (Phases 3–4) let $\tilde{b}_i = b_i / \max_j b_j$ be the BM25 score of s_i normalised by the maximum score in the retrieved set (divisor set to 1 if the maximum is zero).

Hard pruning. Retain s_i if $\tilde{b}_i \geq \theta$, with $\theta \in \{0.5, 0.3\}$. If all sentences are pruned, the top-scoring sentence is retained as a fallback. The earlier Phase 2 implementation instead retains the top-ranked fraction $r \in \{0.7, 0.5, 0.3\}$ of sentences (with a two-sentence floor) under an instruction-wrapped prompt; it is reported separately (Table 2B).

Soft compression. Sentences below the threshold are summarised with T5-small (Raffel et al., 2020) (beam search, max 64 tokens, min 10) rather than dropped entirely (Phase 2).

Hybrid. Sentences with $\tilde{b}_i \geq 0.6$ are kept verbatim; sentences with $0.25 \leq \tilde{b}_i < 0.6$ are T5-compressed; sentences below 0.25 are dropped. This approximates the three-tier structure of AHCC.

4.2 AHCC Architecture

AHCC (Adaptive Hybrid Compression Controller) is built on DistilBERT (Sanh et al., 2019), loaded via the Transformers library (Wolf et al., 2020), with a two-layer classification head appended to the `[CLS]` hidden state: LayerNorm → Dropout(0.2) → Linear(768, 256) → GELU → Dropout(0.2) → Linear(256, 3). The full classifier has about 66.6M parameters.

Each (question, sentence) pair is encoded as:

```
[CLS] [QUERY] {question} [SEP]
      {sentence} [SEP]
```

where `[QUERY]` is an additional special token added to the vocabulary to distinguish the query segment from the evidence segment at the embedding level. The model predicts over three classes: KEEP (label 0), DROP (label 1), EMBED (label 2).

4.3 Automatic Label Generation

Training labels are derived automatically from retrieved sentence lists using two signals: BM25 scores (for term overlap with the question) and cosine similarity from a lightweight sentence encoder (SBERT `all-MiniLM-L6-v2` (Reimers and Gurevych, 2019)):

- **KEEP**: the sentence contains the gold answer span verbatim, *or* its cosine similarity to the question ≥ 0.60.
- **DROP**: cosine similarity < 0.25, *or* BM25 score equals zero (no term overlap).
- **EMBED**: all remaining sentences (partial relevance).

The procedure is applied to the retrieved sentence lists of a pool of 400 questions (200 HotpotQA with 10 sentences each, 200 Natural Questions with 5 each), yielding 3,000 labelled (question, sentence) pairs with the class distribution KEEP: 904, DROP: 573, EMBED: 1,523. We split sentence pairs 75/15/10 (train/validation/test: 2,250/450/300) with stratified sampling. This is a pair-level rather than question-level split. The training partition contains 677 KEEP, 430 DROP, and 1,143 EMBED pairs. Class imbalance is addressed via inverse-frequency class weights in the loss function and a weighted random sampler. The 100 HotpotQA pipeline-evaluation questions belong to the 400-question classifier-development pool (Section 8): they account for 1,000 of the 3,000 labelled pairs (100 questions $\times$ 10 sentences), and their gold answers were used in KEEP label generation. This creates a question-level leakage risk and limits claims about generalisation.

Table 1: Closest related methods. "Training signal" is how the compressor's supervision is obtained.

Method	Granularity	Training signal	Setting studied
LLMLingua	token	none (small-LM perplexity)	reasoning, summarisation, dialogue
LongLLMLingua	token / document	none (question-aware perplexity)	long-context, multi-document QA
LLMLingua-2	token	distilled from GPT-4 outputs	task-agnostic compression
Selective Context	lexical unit	none (self-information)	QA, summarisation, conversation
RECOMP	sentence / abstractive	downstream-LM-derived supervision	open-domain QA, language modelling
FILCO	sentence / span	lexical and information-theoretic overlap	open-domain QA, fact verification, dialogue
EXIT	sentence	trained sentence classifier	single- and multi-hop QA
Provence	sentence	trained pruner (synthetic labels)	QA across domains
CompAct	abstractive, iterative	trained compressor	multi-hop QA
AutoCompressors	soft prompt	unsupervised, fine-tuned LM	in-context learning, retrieval-augmented LM
APOL / AHCC	sentence (3-way)	heuristic labels + self-training	multi-hop QA; pool size, distractors, noise, threshold

4.4 Two-Stage Training

Stage 1 — heuristic supervision. We fine-tune AHCC for 5 epochs using AdamW (Loshchilov and Hutter, 2019) (lr $= 2 \times 10^{-5}$, weight decay $= 0.01$), cross-entropy loss with label smoothing of 0.05, and a linear warmup over 10% of training steps. The best checkpoint is selected by validation macro-F1 (0.575) and evaluated on the test split, where it achieves macro-F1 0.603 against the automatically generated heuristic labels. This is agreement with heuristic labels on a pair-level split; it is not a measure of downstream QA quality or of question-disjoint generalisation.

Stage 2 — self-training. Following classic self-training (Yarowsky, 1995), the Stage 1 model is applied to all retrieved sentences; predictions with maximum class probability ≥ 0.85 yield 956 pseudo-labels, producing a Stage 2 training set of 3,956 pairs. We fine-tune Stage 1 weights for a further 3 epochs at lr $= 5 \times 10^{-6}$. Validation macro-F1 rises from 0.575 to 0.700. Because the pseudo-labelled pool spans all retrieved sentences, and because 3,956 equals 3,000 plus 956 (so the Stage 2 training set appears to include the validation and test pairs), this is a within-pool figure and not a held-out estimate; it is not comparable with the Stage 1 validation score.

4.5 Prompt Assembly

Given classified sentences, the compressed prompt is constructed as: (1) all KEEP sentences concatenated verbatim under a `[Key context]` header, ordered by original document position; (2) the single EMBED sentence with highest cosine similarity to the question, included verbatim under a `[Supporting context]` header; (3) all DROP sentences excluded. Assembly respects a configurable token budget, trimming EMBED content first if the budget is exceeded.

Extractive versus abstractive EMBED assembly. The initial design used T5-small to abstractively summarise all EMBED sentences into the prompt. With the same Stage 1 classifier, the resulting pipeline achieved F1 = 0.137, below the binary KEEP/DROP variant (0.165) and BM25 hard pruning (0.217). Section 7.1 analyses this. Replacing it with extractive top-1 cosine selection gives 0.171 (Stage 1) and 0.181 (Stage 2). All main results use extractive assembly.

Confidence fallback. At inference, if the maximum class probability for a sentence is below the threshold $\tau = 0.60$, the sentence is assigned EMBED regardless of the argmax class, avoiding overly aggressive dropping of low-confidence predictions.

5 Experimental Setup

Dataset. We use HotpotQA (Yang et al., 2018) in the distractor setting, which requires multi-hop reasoning over two supporting paragraphs embedded among distractor paragraphs. We sample 200 instances from the validation set; a fixed subset of 100 is used as the evaluation set, kept consistent across all phases and experimental conditions. Natural Questions (Kwiatkowski et al., 2019) in its open-domain form (NQ-Open; Lee et al. 2019) provides no passages, so the NQ contexts in our

pool are synthetic answer-seeded chunks; NQ contributes 1,000 of the 3,000 classifier-training pairs and no NQ result is reported. *All pipeline comparisons and compression results are computed on the 100 HotpotQA evaluation questions.*

Phases. The experiments were run in five phases. Phases 1 (uncompressed baseline) and 2 (heuristic compression) use the instruction-wrapped template (Comparison B). Phase 3 (AHCC training and pipeline evaluation) and Phase 4 (robustness E1–E2, pool size E3, confidence threshold E4) use the plain template (Comparison A). Phase 5 compares accuracy and prompt length across methods (Section 7.3).

Language model. All evaluations use Mistral-7B-Instruct-v0.2, a release in the Mistral 7B model family (Jiang et al., 2023a), quantised to 4-bit NF4 with double quantisation (Dettmers et al., 2023), loaded on a single NVIDIA Tesla T4 (16 GB VRAM). Greedy decoding with a maximum of 128 new tokens is used throughout. Keeping the LLM and decoding strategy fixed across all conditions isolates the effect of the compression method.

Prompt templates and matched comparisons. We report two internally matched comparisons. In *Comparison A* (Phases 3–4) all methods and the uncompressed baseline share one plain instruction (`Answer the question using only the provided context.`) followed by the question, context, and `Answer:`. In *Comparison B* (Phases 1–2) the baseline and the heuristic methods share an instruction-wrapped (`[INST]`) template with a "be concise" instruction. Each compression method is compared with the baseline of its own comparison.

Retriever. BM25 over the per-question sentence pool. The stored pool contains 10 sentences per HotpotQA question, so $k = 10$ is the full pool and the default setting. The pool-size experiment (Section 6.5) varies $k \in \{5, 10, 15, 20\}$.

Metrics. *Token F1* measures token-level overlap between the predicted and gold answers after normalisation (lowercasing, article and punctuation removal), following Rajpurkar et al. (2016). *Exact Match* (EM) is binary string equality after the same normalisation; with $n = 100$, one question corresponds to 0.010 EM. *Efficiency Score* is the per-sample average of $\mathrm{F1}/(\text{tokens} \times 10^{-3})$, capturing quality per unit of prompt length; as a mean of ratios it differs from mean-F1 over mean-tokens and favours very short prompts. *Compression Ratio* is $1 - (\text{compressed tokens}/\text{original tokens})$, computed per sample against the original sentence tokens and averaged, so it differs from one minus the ratio of mean prompt lengths (by up to about six points in Table 2A; we did not decompose this gap). All token counts are approximations ($1.3\times$ whitespace words), not tokenizer counts.

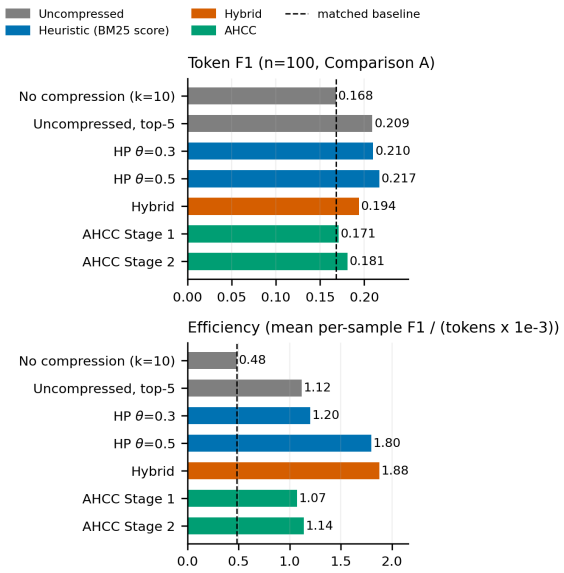


Figure 1: Token F1 (left) and Efficiency Score (right) for Comparison A (same run, same prompt template, $n = 100$). The dashed line marks the matched uncompressed baseline (F1 0.168; efficiency 0.481). AHCC Stage 2 is the proposed method. In the figure legend, "Hard prune (moderate)" is HP θ=0.5 and "Hard prune (aggressive)" is HP θ=0.3 of Table 2.

Uncertainty. We report descriptive normal-approximation 95% intervals $\pm 1.96\,\hat{\sigma}/\sqrt{n}$ from the logged per-sample standard deviations (Appendix B). These are marginal intervals, not paired confidence intervals for method differences.

Reproducibility. Seeds are fixed to 42 for data splits and training. Hyperparameters are given in Appendix A.

6 Results

6.1 Main Results

In both matched comparisons, every compression method in Table 2 has a higher observed Token F1 than its uncompressed baseline while using fewer tokens (Table 2, Figure 1). This does not extend to every AHCC variant or condition: two ablation

Table 2: Main results (HotpotQA evaluation set, 100 samples, BM25 $k = 10$). Tok. = mean approximate prompt tokens; Comp. = mean per-sample fraction of original tokens removed; Eff. = mean per-sample efficiency. Each block is compared with its own baseline (first row of the block); the top-5 row is an uncompressed reference. † = proposed method. **Bold** = best per column among compression methods in a block.

Method	F1	EM	Tok.	Comp.	Eff.
A. Plain template (Phases 3–4)					
No compression (k=10)	0.168	0.010	354.5	0.000	0.481
Uncompressed, top-5	0.209	0.030	193.7	—	1.118
HP θ=0.3	0.210	0.030	229.8	0.266	1.202
HP θ=0.5	**0.217**	**0.040**	152.9	0.512	1.800
Hybrid	0.194	**0.040**	**123.9**	**0.603**	**1.877**
AHCC Stage 1	0.171	**0.040**	182.4	0.460	1.072
AHCC Stage 2†	0.181	0.030	176.9	0.473	1.139
B. Instruction-wrapped template (Phases 1–2)					
No compression	0.116	0.030	390.9	0.000	0.297
HP top 70%	0.197	0.030	270.0	0.313	0.732
HP top 50%	**0.215**	**0.040**	208.1	0.508	1.034
HP top 30%	0.196	0.020	**145.2**	**0.703**	1.345
Hybrid	0.195	0.020	152.6	0.679	**1.358**
Soft compress (T5)	0.165	0.010	271.7	0.306	0.611

HP = Hard Prune (threshold θ on normalised BM25 in A; top fraction of ranked sentences in B).

variants (Section 6.3) and AHCC under tail-noise injection (Section 6.4) fall below the uncompressed prompt.

Comparison A. Against the baseline (F1 0.168, 354.5 tokens), the compression methods use 35–65% fewer tokens at F1 0.171–0.217. AHCC Stage 2 reaches 0.181 with 176.9 tokens (50% fewer; 47.3% compression by the per-sample definition). Hard pruning at θ=0.5 obtains the highest F1 (0.217, 152.9 tokens) and Hybrid the highest efficiency (1.877, 123.9 tokens). The five-sentence uncompressed reference is also strong (0.209 at 193.7 tokens); hard pruning at θ=0.5 exceeds it with 21% fewer tokens, and AHCC Stage 2 (0.181) sits between this reference and the full-pool baseline.

Comparison B. Under the instruction-wrapped template the baseline scores 0.116, and the compression methods reach 0.165–0.215 (+0.049 to +0.099) with 31–70% compression, with moderate hard pruning again the most accurate. T5 soft compression is the weakest here (0.165); it also requires an additional neural pass, whose latency we did not measure.

Reading the differences. With per-question standard deviations of 0.20–0.26, the 95% interval on each F1 is roughly ±0.04–0.05 (Appendix B). The observed token reductions are substantial in this sample. However, marginal intervals do not establish the significance of paired method differences, and the evaluation design further limits inference. EM values of 0.010–0.040 correspond to one to four correct answers out of 100, so EM differences between methods are one to three questions. The baseline changes by 0.052 F1 between the two templates (0.168 vs. 0.116), larger than most between-method gaps within a comparison, which shows that this metric is sensitive to prompt format. We therefore describe the F1 ranking as observed rather than statistically reliable, avoid claiming that compression preserves accuracy in general, and do not pool the two comparisons.

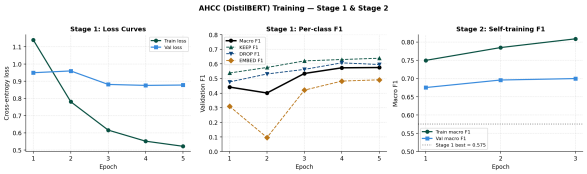


Figure 2: AHCC (DistilBERT) training dynamics. *Left*: Stage 1 loss curves; train loss decreases steadily and validation loss plateaus after epoch 3. *Centre*: per-class validation F1 across Stage 1 epochs. *Right*: Stage 2 macro-F1 during self-training; dotted line marks the Stage 1 best (0.575).

Table 3: AHCC classifier agreement with the heuristic labels. Stage 1 trains on 2,250 of the 3,000 labelled pairs. Stage 2 adds 956 pseudo-labels with confidence ≥ 0.85; its count (3,956) equals 3,000 + 956, so it appears to include the validation and test pairs. Val-F1 is validation macro-F1 (used for checkpoint selection); Stage 1 macro-F1 on the pair-level test split is 0.603. K/D/E-F1: per-class *test* F1 for KEEP / DROP / EMBED (Stage 1). The Stage 2 value is a within-pool figure.

Stage	Train pairs	Val-F1	K-F1	D-F1	E-F1
Stage 1	2,250	0.575	0.631	**0.667**	0.513
Stage 2	3,956	0.700	—	—	—

6.2 AHCC Classifier Performance

Table 3 and Figure 2 show the classifier's agreement with the heuristic labels at both stages. Stage 1 achieves macro-F1 of 0.603 on the pair-level test split, with DROP as the strongest class (0.667) and EMBED as the weakest (0.513), possibly reflecting the ambiguity of the intermediate relevance class. These scores measure agreement with automatically generated labels rather than human judgements of evidence usefulness. Validation

Table 4: Ablation study (Comparison A). ΔF1 is measured against the matched no-compression baseline (0.168), so every configuration shares a single anchor. † = proposed configuration.

System	F1	ΔF1	Comp.
No compression (baseline)	0.168	—	0.00
Hard prune (θ=0.5)	0.217	+0.049	0.51
AHCC KEEP+DROP only	0.165	−0.003	—
AHCC S1 + T5 abstractive	0.137	−0.031	0.44
AHCC S1 + extractive EMBED	0.171	+0.003	0.46
AHCC S2 + extractive EMBED†	0.181	+0.013	0.47

macro-F1 rises by 0.125 from Stage 1 to Stage 2, but this is a within-pool figure; the downstream gain of Stage 2 over Stage 1 is only 0.010 F1 (0.171 to 0.181), small relative to sampling error.

6.3 Ablation Study

Table 4 compares the AHCC design variants. Two of them (KEEP+DROP only, and T5 abstractive EMBED) score below the uncompressed baseline.

Three-class versus binary. Removing the EMBED class and using binary KEEP/DROP gives F1 0.165 versus 0.171 with the three-class policy, a difference too small to interpret at this sample size.

Abstractive versus extractive EMBED. The largest gap among the AHCC variants is between T5-abstractive EMBED (F1 = 0.137) and extractive top-1 EMBED (F1 = 0.171). The classification decisions are identical (both use the Stage 1 model); only the assembly differs. This is the main finding of our study (Section 7.1), but at 0.034 it is comparable with the marginal 95% half-widths and has not been tested for significance.

Stage 2 self-training. Self-training on pseudo-labels is followed by an F1 increase of 0.010 (0.171 → 0.181) without additional human annotation. This difference is small relative to sampling error and the Stage 2 data include the evaluation questions, so we do not conclude that self-training improves QA quality.

6.4 Robustness Evaluation

E1: Distractor injection. As distractor sentences are injected (5, 10, 20 per sample), each of the compressors in Table 5 stays above the uncompressed prompt at every level (AHCC: 0.167, 0.169, 0.152 vs. 0.165, 0.150, 0.145; the margin is 0.002 at +5), and hard pruning at θ=0.5 is the most accurate method throughout, rising to 0.236

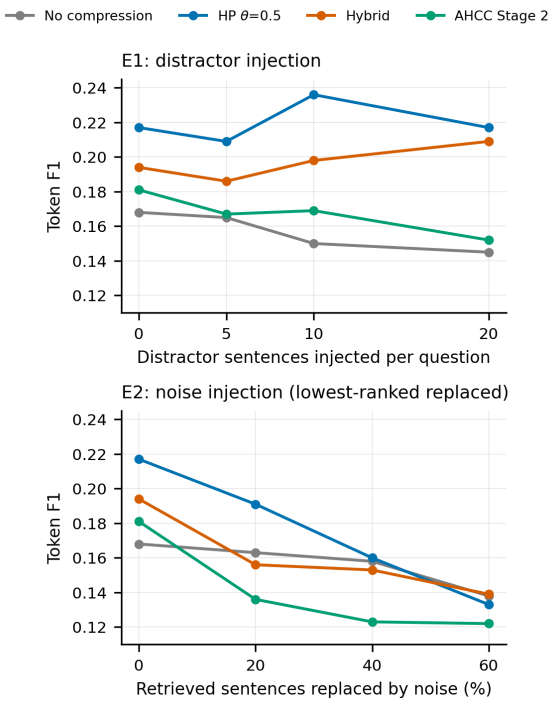


Figure 3: Robustness evaluation. *Left (E1)*: Token F1 as distractor sentences from other questions are injected per sample. *Right (E2)*: Token F1 as a fraction of retrieved sentences (the lowest-ranked ones) is replaced by unrelated sentences. In the legend, "Hard prune (moderate)" is HP θ=0.5.

at 10 distractors. This is consistent with BM25 continuing to rank supporting sentences above distractors from other questions, although we did not test this directly. Prompt length stays small: AHCC grows from 184 to 203 tokens (+10%, the smallest relative growth), hard pruning from 156 to 219 (+40%), and Hybrid from 129 to 159 (+23%), compared with 485 to 865 tokens for the uncompressed prompt.

E2: Noise injection. In E2 the replaced sentences occupy the lowest BM25 ranks. The uncompressed prompt degrades from 0.168 (no noise) to 0.163, 0.158, and 0.138 at 20%, 40%, and 60% noise, whereas the compression methods start from different clean values and fall further: hard pruning from 0.217 to 0.133, Hybrid from 0.194 to 0.139, and AHCC from 0.181 to 0.122. AHCC is below the uncompressed prompt at all three noise levels; Hybrid is below it at 20% (0.156 vs. 0.163) and 40% (0.153 vs. 0.158); and hard pruning is below it at 60% (0.133 vs. 0.138). Only hard pruning at 20% and 40% noise (0.191 and 0.160) exceeds the uncompressed prompt, narrowly at 40%. At 60%

Table 5: Robustness results (Token F1; Comparison A template). E1: distractor sentences from other samples injected. E2: fraction of retrieved sentences replaced with unrelated sentences (the replaced sentences are the lowest-ranked by BM25). **Bold** = best per column.

	E1: Distractors			E2: Noise fraction		
Method	**+5**	**+10**	**+20**	**20%**	**40%**	**60%**
No compression	0.165	0.150	0.145	0.163	0.158	0.138
HP θ=0.5	**0.209**	**0.236**	**0.217**	**0.191**	**0.160**	0.133
Hybrid	0.186	0.198	0.209	0.156	0.153	**0.139**
AHCC S2†	0.167	0.169	0.152	0.136	0.123	0.122

Table 6: E3: Token F1 / mean approximate tokens at $k = 5$ and $k = 10$. Results at $k = 15$ and $k = 20$ are identical to $k = 10$ for every method.

Method	$k = 5$	$k = 10$
No compression	0.209 / 194	0.168 / 354
HP θ=0.5	0.213 / 116	0.217 / 153
HP θ=0.3	0.212 / 158	0.210 / 230
Hybrid	0.198 / 102	0.194 / 124
AHCC S2†	0.178 / 129	0.181 / 177

noise all four methods lie within 0.02 F1 of each other. At 20–40% noise AHCC uses about 55–58% fewer tokens than the uncompressed prompt. E2 therefore marks a boundary of the accuracy benefit: when noise is concentrated in the tail of the ranking, the compressors tested here have less to remove and mostly do not improve on the uncompressed prompt. AHCC's compression ratio rises from 0.47 to 0.59 as noise grows, more than hard pruning (0.51 to 0.55) or Hybrid (0.60 to 0.61), but we did not measure which sentences were removed, so we do not claim noise identification.

6.5 Retrieval Pool Size

Figure 4 and Table 6 show F1 and prompt length as k varies. Because the stored pool contains ten sentences per question, $k = 15$ and $k = 20$ return the same context as $k = 10$ for every method, so the informative contrast is $k = 5$ versus $k = 10$. Going from five to ten retrieved sentences, the uncompressed prompt loses about 20% of its F1 (0.209 to 0.168), while the F1 of every compression method changes by 0.004 or less in absolute value (rounded values). Over this range, compressed pipelines are therefore less sensitive to pool size than the uncompressed prompt, at roughly half the prompt length or less at $k = 10$. We did not test whether this extends to larger pools. Truncating the uncompressed prompt to its top five sentences already achieves F1

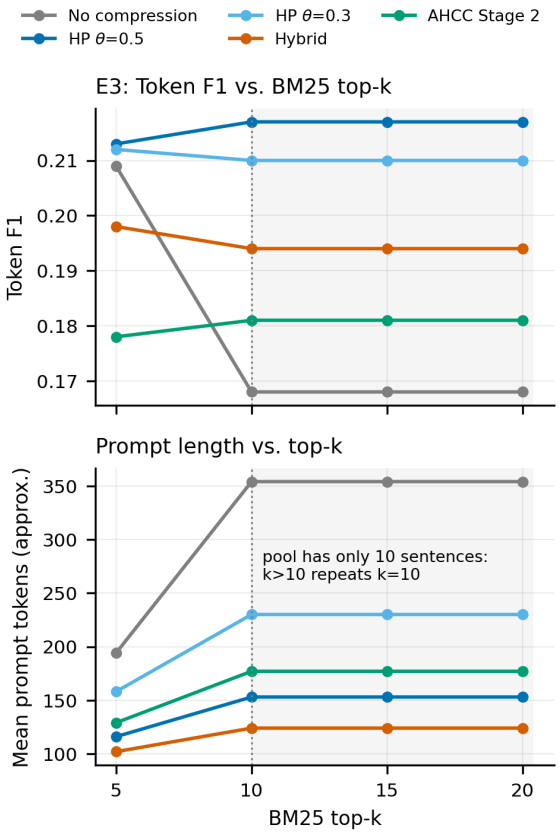


Figure 4: E3: Token F1 (left) and Efficiency Score (right) as BM25 top-k varies from 5 to 20. The stored pool has 10 sentences per question, so $k \in \{10, 15, 20\}$ gives the same context for every method. Dotted line marks the default $k = 10$. In the legend, "Hard prune (moderate)" is HP θ=0.5.

0.209, above AHCC at either depth (0.178, 0.181), which makes it a useful reference point. Testing larger pools requires re-retrieving from a larger corpus, which we leave to future work.

6.6 Confidence Threshold Analysis

Figure 5 shows F1, compression, and efficiency as τ is swept from 0.50 to 0.80 (Appendix C). Token F1 is highest at $\tau = 0.60$ (0.181), the default setting used throughout this paper, and efficiency is highest at $\tau = 0.80$ (compression = 63.0%, efficiency = 1.463, F1 = 0.169). Compression rises monotonically with τ (41.0% to 63.0%, mean prompt size 198.5 to 123.9 tokens), whereas F1 is non-monotonic: it stays between 0.167 and 0.181, at or above the matched uncompressed F1 (0.168) for $\tau \geq 0.55$. The sweep shows that changing a single confidence threshold after training moves the compression level over this range without a clear F1 trend; the F1 variation is within the marginal

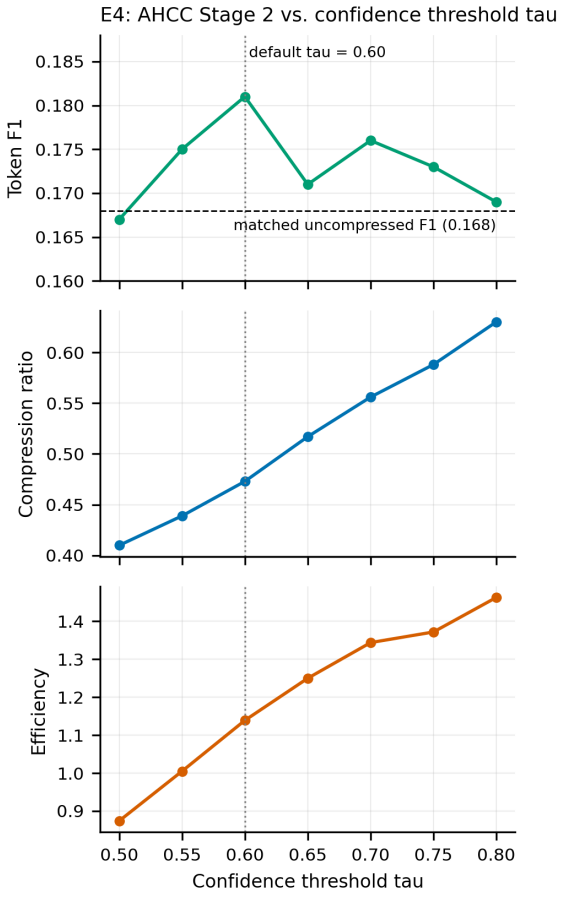


Figure 5: E4: AHCC confidence threshold trade-off. Left: F1 and compression ratio vs. τ (dashed red line: matched uncompressed F1). Right: efficiency vs. τ.

intervals of Appendix B. The sweep uses the same evaluation questions that overlap with the training pool, and $\tau = 0.60$ is also its F1-maximising point, so the reported 0.181 should not be read as an unbiased estimate, nor as evidence that the same operating points will hold on unseen questions.

7 Analysis

7.1 Finding 1: Assembly Matters for a Fixed Classifier

The most consequential observation in this study is the gap between classifier agreement with heuristic labels and downstream QA performance when abstractive context assembly is employed. Stage 1 AHCC achieves macro-F1 of 0.603 on the pair-level test split against its heuristic labels, yet when EMBED sentences are compressed abstractively with T5-small, the QA pipeline produces Token F1 of only 0.137. This is lower than the binary KEEP/DROP variant (0.165), which makes no use of the EMBED class, and than BM25 hard pruning (0.217). Replacing T5-small with extractive top-1 selection — keeping the single EMBED sentence most similar to the question by cosine similarity, verbatim — raises pipeline F1 to 0.171 with the same classifier and the same training.

This experiment varies the assembly step with the classifier held fixed; it does not vary classifier accuracy, so it shows that assembly can dominate the observed pipeline F1, not that accuracy and pipeline quality are uncorrelated across classifiers. The 0.034 gap is comparable with the marginal 95% half-widths (about 0.04–0.05) and no paired test was run.

We hypothesise that T5-small, limited to 64 output tokens while summarising several EMBED sentences, drops answer-bearing spans, particularly for multi-hop questions where the answer is distributed across sentences. We did not measure how often the gold answer is present in the EMBED input but absent from the T5 output, so this is an untested hypothesis. Extractive top-1 also discards all but one EMBED sentence, so its benefit may reflect avoiding destructive rewriting rather than retaining more evidence.

A tentative methodological implication is that classification-level agreement and pipeline-level quality should be reported separately in RAG compression studies, since a compressor with reasonable sentence-level F1 may still degrade downstream QA if its assembly step loses information.

7.2 Finding 2: BM25 Thresholding versus AHCC on HotpotQA

BM25-threshold pruning attains higher raw Token F1 than AHCC in clean conditions (0.217 vs. 0.181). The 0.036 difference is smaller than the marginal 95% half-widths (about 0.05); without paired per-question predictions we cannot say whether it is significant. One plausible explanation, which we did not test, is a structural property of HotpotQA's distractor setting: supporting paragraphs tend to be lexically similar to the question, so answer-bearing sentences may receive higher BM25 scores than distractor sentences, and score thresholding could act as an effective proxy for answer relevance. Our training labels also derive partly from BM25 and SBERT similarity, so AHCC may be learning a related signal. A related hypothesis is that a learned controller would be comparatively more useful where BM25 scores are less predictive of answer relevance, such as paraphrase-intensive

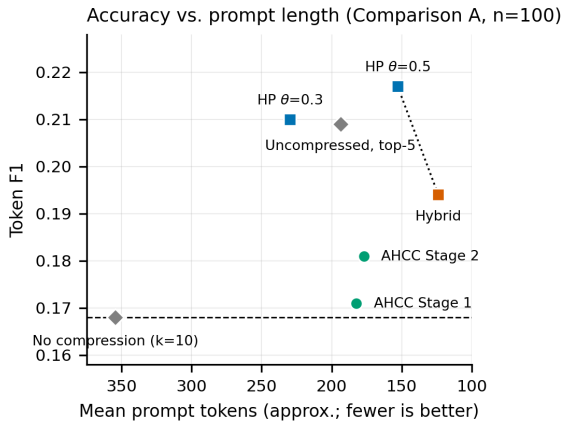


Figure 6: Token F1 versus mean prompt tokens (Comparison A, $n = 100$). Circles: AHCC; squares: heuristic methods; diamonds: uncompressed prompts. Up and to the right is better; the dotted line connects the non-dominated points. In the legend, "Hard prune (moderate)" is HP θ=0.5 and "Hard prune (aggressive)" is HP θ=0.3.

datasets or dense retrieval pipelines. We have no evidence for or against this; evaluating it across a broader set of datasets and retrieval configurations is an important direction for future work.

7.3 Accuracy versus Prompt Length

Figure 6 places each method on the F1–prompt-length plane. Hard pruning at θ=0.5 and Hybrid define the frontier. AHCC Stage 2 reaches F1 0.181 at 177 tokens, above the full-pool baseline at half its length, and, unlike the threshold rules, it exposes the classifier–assembly interaction of Section 7.1 and a post-training threshold (Section 6.6) that moves it along the compression axis. Its value in this study lies in that analysis rather than in outperforming the heuristics on HotpotQA.

8 Limitations

Training pool and held-out evaluation. AHCC's labels were generated from a 400-question pool that includes the 100 HotpotQA evaluation questions (1,000 of the 3,000 labelled pairs), and the labelled pairs were split at the pair level; KEEP labels use gold-answer containment, so answer locations for the evaluation questions are encoded in the training labels. The classifier never sees LLM outputs, and the heuristic baselines have no trained component, but the AHCC results should be read as in-pool rather than held-out. The Stage 2 training set appears to include the validation and test pairs, so the Stage 2 validation score is likewise not held out. Evaluating on questions excluded from label generation, self-training, and checkpoint selection is the most important next step.

Scale and statistics. All pipeline evaluations use 100 HotpotQA questions with one reader. Intervals of about $\pm$0.05 F1 leave most pairwise differences among compression methods unresolved; a paired per-question bootstrap (Berg-Kirkpatrick et al., 2012) and larger evaluation sets would tighten them. EM differences correspond to one to three questions.

Baselines and datasets. We compare against our own heuristic baselines and do not re-run published compressors such as LLMLingua-2, RECOMP, EXIT, or Provence. HotpotQA's distractor setting may also favour BM25-based methods (Section 7.2); evaluation on Natural Questions with real passages, TriviaQA, and single-hop datasets is an important extension.

Metric and prompt templates. Absolute scores are low (F1 0.12–0.22, EM 0.01–0.04), and Token F1 depends on answer length and format; we did not analyse answer length, so some F1 differences may reflect changes in answer verbosity rather than evidence quality. The two comparisons use different templates (Section 5), the baseline differs by 0.052 F1 between them, and AHCC appears only in Comparison A. We compare each method with the baseline of its own comparison and do not pool them. Token counts are approximations. The recall of supporting facts in the stored pool and in each compressor's output was not measured.

Retrieval pool and noise. The pool contains 10 sentences per question, so behaviour for $k > 10$ is untested. In E2 the replaced sentences are the lowest-ranked ones, which favours methods that already discard low-ranked sentences, and E1/E2 distractors were drawn with a per-process seed, so exact draws may differ across runs. The threshold sweep (E4) reuses the evaluation questions, so the chosen τ is not independently validated.

Language model and labels. Only Mistral-7B-Instruct-v0.2 at 4-bit quantisation is evaluated; larger, full-precision, or differently tuned models may differ. AHCC labels derive from BM25, SBERT, and gold-answer heuristics and inherit their biases; classifiers trained on downstream QA supervision, as in RECOMP (Xu et al., 2024), may generalise more broadly.

Untested explanations. The explanations offered for the T5 assembly gap (Section 7.1) and for the strength of BM25 thresholding (Section 7.2) are hypotheses; neither mechanism was directly tested.

9 Conclusion

We introduced APOL, an exploratory evaluation protocol for sentence-level prompt compression in retrieval-augmented QA, and AHCC, a learned compression controller trained without human annotation. Across five experimental phases, we observed the following:

(1) In this 100-question sample, every compression method in the main comparisons has a higher observed Token F1 than its corresponding uncompressed baseline within each prompt-template comparison, while using fewer tokens. In Comparison A, AHCC scores 0.181 versus 0.168 for the baseline (47.3% mean per-question compression), while BM25-threshold pruning scores 0.217 (51.2% compression). Two AHCC ablation variants (0.165 and 0.137) and AHCC under tail-noise injection fall below the uncompressed prompt. None of these differences has been established as statistically significant.

(2) Between $k = 5$ and $k = 10$ in the stored pool, each compression method changes by 0.004 F1 or less, while the uncompressed baseline falls from 0.209 to 0.168. The pool does not support conclusions about $k > 10$.

(3) With the classifier fixed, pipeline F1 is 0.137 with T5 assembly and 0.171 with extractive top-1 selection, indicating that the assembly step can dominate pipeline quality; the mechanism is untested.

(4) The AHCC confidence threshold moved compression from 41.0% to 63.0% without retraining, with Token F1 varying non-monotonically between 0.167 and 0.181.

These exploratory findings illustrate the trade-off between prompt length and observed answer quality in this setup and point to a possible design risk when classifier outputs are converted into compressed context. Because the evaluation questions belong to the classifier-development pool, the results do not establish generalisation to unseen questions or superiority over established compressors. Held-out evaluation of the classifier, comparison with published compressors, paired significance testing, and larger retrieval pools are the natural next steps.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. 2012. An empirical investigation of statistical significance in NLP. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005. Association for Computational Linguistics.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846. Association for Computational Linguistics.

Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. 2025. Provence: Efficient and robust context pruning for retrieval-augmented generation. In *International Conference on Learning Representations*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Taeho Hwang, Sukmin Cho, Soyeong Jeong, Hoyun Song, SeungYoon Han, and Jong C. Park. 2025. EXIT: Context-aware extractive compression for enhancing retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4895–4924. Association for Computational Linguistics.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880. Association for Computational Linguistics.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud,

Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023a. Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023b. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376. Association for Computational Linguistics.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677. Association for Computational Linguistics.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466.

Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent retrieval for weakly supervised open domain question answering. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6086–6096, Florence, Italy. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6354. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Maxime Louis, Hervé Déjean, and Stéphane Clinchant. 2025. PISCO: Pretty simple compression for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15506–15521. Association for Computational Linguistics.

Lvzhou Luo, Yixuan Cao, and Ping Luo. 2025. AttnComp: Attention-guided adaptive context compression for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8456–8472. Association for Computational Linguistics.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822. Association for Computational Linguistics.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981. Association for Computational Linguistics.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392. Association for Computational Linguistics.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR.

Zhiruo Wang, Jun Araki, Zhengbao Jiang, Md Rizwan Parvez, and Graham Neubig. 2023. Learning to filter context for retrieval-augmented generation. *arXiv preprint arXiv:2311.08377*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics.

David Yarowsky. 1995. Unsupervised word sense disambiguation rivaling supervised methods. In *33rd Annual Meeting of the Association for Computational Linguistics*, pages 189–196. Association for Computational Linguistics.

Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. 2024. CompAct: Compressing retrieved documents actively for question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21424–21439. Association for Computational Linguistics.

A Hyperparameter Configuration

Table 7: Complete hyperparameter configuration.

Parameter	**Value**
Data	
Labelled questions	400 (200 HotpotQA, 200 NQ)
Labelled pairs	3,000
Train / val / test	2,250 / 450 / 300
Sentences per question	10 (HotpotQA), 5 (NQ)
Retrieval	
BM25 top-k (default)	10
AHCC *classifier*	
Base model	distilbert-base-uncased
Max sequence length	256
Dropout	0.2
Stage 1 epochs	5
Stage 1 learning rate	2×10^{-5}
Stage 1 weight decay	0.01
Stage 1 label smoothing	0.05
Stage 1 warmup ratio	0.10
Stage 2 epochs	3
Stage 2 learning rate	5×10^{-6}
Pseudo-label confidence threshold	0.85
Inference confidence threshold τ	0.60
LLM inference	
Model	Mistral-7B-Instruct-v0.2
Quantisation	4-bit NF4, double quant
Decoding	Greedy
Max new tokens	128
Batch size	4
Phase 4 experiments	
E1 distractor counts	5, 10, 20
E2 noise fractions	0.0, 0.2, 0.4, 0.6
E3 top-k values	5, 10, 15, 20
E4 confidence thresholds	0.50 to 0.80 (step 0.05)
Random seed (splits, training)	42

B Confidence Intervals

We report normal-approximation 95% intervals, $\pm 1.96\,\hat{\sigma}/\sqrt{100}$, computed from the per-sample standard deviations of Token F1 logged for Comparison A ($n = 100$):

All methods answer the same 100 questions, so paired per-question analysis would be preferable for comparing methods. The per-question predictions needed to compute paired bootstrap intervals are not included in the supplied materials; we therefore report these marginal intervals only and do not claim statistically significant differences.

C Confidence Threshold Sweep

Table 9 reports the full E4 operating curve for AHCC Stage 2 with extractive assembly, from which the $\tau = 0.60$ (peak F1) and $\tau = 0.80$ (peak efficiency) operating points are drawn.

Table 8: Token F1 with 95% half-widths (Comparison A).

Method	**F1**	$\hat{\sigma}$	$\pm$**95%**
No compression (k=10)	0.168	0.202	0.040
Uncompressed, top-5	0.209	0.251	0.049
HP θ=0.3	0.210	0.252	0.049
HP θ=0.5	0.217	0.263	0.052
Hybrid	0.194	0.256	0.050
AHCC Stage 2	0.181	0.241	0.047

Table 9: E4 confidence threshold sweep (AHCC Stage 2, extractive). Token F1 peaks at $\tau = 0.60$; efficiency and compression increase monotonically with τ.

τ	**F1**	**Comp.**	**Eff.**
0.50	0.167	0.410	0.874
0.55	0.175	0.439	1.005
0.60	**0.181**	0.473	1.139
0.65	0.171	0.517	1.250
0.70	0.176	0.556	1.344
0.75	0.173	0.588	1.372
0.80	0.169	0.630	**1.463**